Full-Stack Laravel Development

Professional 3-Month Course Lecture Sheet

HTML • CSS • JavaScript • Tailwind CSS • PHP • OOP • Laravel • MySQL • PostgreSQL • React • Inertia.js • GitHub • CI/CD • Server Configuration • Live Deployment

Duration	Teaching	Assessment	Learning Model
3 Months	30 Classes	6 Classes	Project-Driven

Student Lecture Sheet: Every class includes topic titles, concise topic explanations, and a practical home-task problem. The course progressively builds toward a production-style Laravel + React + Inertia capstone application.

Student Success Roadmap — How to Get the Best Result

01 • Learn
Understand the concept before copying code.

02 • Practice
Complete the class exercise and home task.

03 • Build
Add each skill to the progressive capstone.

04 • Ship
Test, document, deploy, and present the final app.

Student standard: keep your GitHub active, debug your own errors, ask questions, review weekly, and never leave a practical task unfinished.

Program Overview

This course is designed to take students from web fundamentals to building professional Laravel applications using React and Inertia. The training follows an industry-oriented, project-driven structure with hands-on practice in every class.

1. Learning Outcomes

- Build semantic, responsive interfaces with HTML, CSS, JavaScript, and Tailwind CSS.
- Write structured PHP using OOP principles and clean coding practices.
- Design relational databases and write practical SQL in MySQL and PostgreSQL.
- Build Laravel applications with routing, controllers, Eloquent, validation, authentication, and authorization.
- Create React frontends and connect them to Laravel using Inertia.js.
- Prepare real-world projects with testing, security, documentation, GitHub workflow, CI/CD, server configuration, and live deployment.

Student Learning Journey

1. **Foundation**
   HTML, CSS, JavaScript, responsive UI, and browser-side problem solving.
2. **Backend & Data**
   PHP, OOP, MySQL, PostgreSQL, Composer, and clean server-side structure.
3. **Laravel Engineering**
   CRUD, authentication, authorization, storage, APIs, testing, and performance.
4. **Modern Full Stack**
   React + Inertia components, forms, state, reusable UI, and integrated workflows.
5. **Production Delivery**
   GitHub, CI/CD, server configuration, live deployment, monitoring, and handover.

Target outcome: by the final assessment, you should be able to explain, build, test, deploy, and demonstrate your own Laravel full-stack application.

2. Course Format

Duration	3 Months
Teaching Sessions	30 Classes
Assessment Classes	6 Classes
Delivery Style	Project-Driven
Recommended Class	2.5-3 Hours

3. Technology Baseline

HTML5 · Modern CSS · JavaScript ES2026 · Tailwind CSS 4 · PHP 8.5 · Laravel 13 · MySQL 8.4 · PostgreSQL 18 · React 19 · Inertia.js 3 · GitHub · CI/CD · Server Configuration · Live Deployment

4. Progressive Capstone

Students progressively build a **Team Project & Task Management System** with authentication, roles, projects, tasks, comments, attachments, filters, dashboards, notifications, automated tests, GitHub workflow, CI/CD, server configuration, live deployment, and deployment-ready documentation. This keeps every phase connected to a real application instead of isolated examples.

Phase 1 • Classes 01-05

Phase 1 — Frontend Foundations

Build the core web foundation with semantic structure, styling, responsiveness, and JavaScript basics.

Class 01

Web Fundamentals & Developer Setup

Topic 1 — Internet, Browser & HTTP Basics: Understand how websites work, how browsers communicate with servers, and how the HTTP request-response cycle connects frontend and backend development.

Topic 2 — Development Environment Setup: Install VS Code, Git, Node.js, PHP, and Composer; organize a clean workspace and understand the role of version control.

Home Task Problem: Set up your development environment and build a one-page personal profile with hero, about, skills, and contact sections.

Class 02

Semantic HTML Foundations

Topic 1 — Document Structure & Core Elements: Learn the modern HTML document structure, headings, paragraphs, lists, links, images, tables, forms, and commonly used attributes.

Topic 2 — Semantic & Accessible Markup: Use header, nav, main, section, article, aside, and footer correctly to improve structure, accessibility, SEO, and maintainability.

Home Task Problem: Build a multi-section portfolio page using only semantic HTML, including navigation, projects, services, and a contact form.

Class 03

Modern CSS Basics

Topic 1 — Selectors, Cascade & Box Model: Work with selectors, inheritance, specificity, spacing, sizing, colors, typography, borders, backgrounds, and the CSS box model.

Topic 2 — Visual Styling Principles: Use display, positioning, reusable classes, shadows, states, and consistent spacing to create a polished interface.

Home Task Problem: Style the Class 02 portfolio into a professional landing page with a consistent visual system and clean component spacing.

Class 04

Responsive Layout with Flexbox & Grid

Topic 1 — Responsive Design Strategy: Use mobile-first thinking, breakpoints, fluid widths, media queries, and responsive spacing so pages adapt across devices.

Topic 2 — Flexbox & CSS Grid: Use Flexbox for alignment and one-dimensional layouts, and CSS Grid for structured multi-column page sections.

Home Task Problem: Create a responsive business landing page with hero, feature cards, pricing, testimonials, and footer for mobile and desktop.

Class 05

JavaScript Core Concepts

Topic 1 — Programming Fundamentals: Practice variables, data types, operators, conditions, loops, functions, arrays, objects, and reusable problem-solving patterns.

Topic 2 — Basic DOM Interaction: Select elements, listen to events, update content, toggle classes, and connect simple JavaScript logic to page interactions.

Home Task Problem: Add an interactive calculator, FAQ accordion, or validated mini form to one of your existing frontend pages.

Phase 2 • Classes 06-10

Phase 2 — JavaScript, Tailwind & PHP Core

Move from frontend interactivity into utility-first styling and structured PHP programming fundamentals.

Class 06

Modern JavaScript & DOM

Topic 1 — Modern ES Features: Use let/const, template literals, destructuring, spread syntax, arrow functions, and modules to write cleaner JavaScript.

Topic 2 — DOM Events & Form Handling: Handle element selection, event listeners, event delegation, dynamic UI updates, and frontend form interactions.

Home Task Problem: Build an interactive todo list with add, edit, complete, delete, filter, and empty-state behavior.

Class 07

Async JavaScript & APIs

Topic 1 — Promises, Async/Await & Fetch: Understand asynchronous execution, consume APIs with fetch, parse JSON, and structure async code with clear error handling.

Topic 2 — Loading, Error & Data States: Render loading indicators, empty states, error messages, and remote data without creating confusing user experiences.

Home Task Problem: Use a public API to build a searchable card/list interface that handles loading, success, empty, and error states.

Class 08

Tailwind CSS & UI Workflow

Topic 1 — Utility-First Styling: Use Tailwind utilities for spacing, colors, typography, borders, layout, responsive variants, and interaction states.

Topic 2 — Reusable UI Composition: Build cards, buttons, forms, badges, navigation, and layout sections with reusable class patterns and minimal custom CSS.

Home Task Problem: Rebuild a modern responsive landing page in Tailwind CSS with navbar, cards, CTA section, forms, and footer.

Class 09

PHP Fundamentals

Topic 1 — PHP Syntax & Server-Side Logic: Practice variables, arrays, conditions, loops, functions, includes, superglobals, and the role of PHP in server-rendered applications.

Topic 2 — Forms & Validation Basics: Read GET/POST input, validate required values, safely display output, and structure basic request-processing logic.

Home Task Problem: Build a PHP contact form that validates name, email, subject, and message before rendering a success or error response.

Class 10

OOP with PHP

Topic 1 — Classes, Objects & Methods: Understand classes, constructors, properties, methods, visibility, encapsulation, and how objects collaborate to model application logic.

Topic 2 — Interfaces, Traits & Exceptions: Use inheritance carefully, apply interfaces and traits, organize namespaces, and use exceptions for predictable error handling.

Home Task Problem: Create Student, Course, and Enrollment classes and print enrollment summaries using object methods rather than global procedural logic.

Phase 3 • Classes 11-15

Phase 3 — Databases & Laravel Foundations

Bridge structured PHP knowledge into practical SQL and modern Laravel application architecture.

Class 11

Composer & Clean PHP Project Structure

Topic 1 — Composer & Autoloading: Use Composer for package management, PSR-4 autoloading, dependency installation, and maintainable project bootstrapping.

Topic 2 — Clean Project Organization: Separate reusable classes, configuration, business logic, and presentation so a PHP project remains understandable as it grows.

Home Task Problem: Build a mini PHP application using Composer autoloading and separate business logic from page rendering.

Class 12

MySQL Database Design

Topic 1 — Relational Schema Design: Create tables with appropriate data types, primary keys, foreign keys, constraints, and relationships for a real business system.

Topic 2 — Practical SQL Operations: Write INSERT, SELECT, UPDATE, DELETE, JOIN, ORDER BY, GROUP BY, aggregate, and LIMIT queries for practical reporting.

Home Task Problem: Design a MySQL schema for users, projects, tasks, and statuses, then write at least five useful business queries.

Class 13

PostgreSQL & Advanced SQL

Topic 1 — PostgreSQL Core Concepts: Understand PostgreSQL workflow, constraints, sequences/identity, views, useful data types, and an introduction to JSON/JSONB.

Topic 2 — Advanced Query Thinking: Practice grouping, filtering, normalization, transactions, and database-specific features while comparing PostgreSQL with MySQL.

Home Task Problem: Recreate part of your task system in PostgreSQL and write a short comparison plus five advanced SQL queries.

Class 14

Laravel Introduction & Project Setup

Topic 1 — Laravel Structure & MVC: Understand Laravel folders, MVC responsibilities, routes, controllers, Blade views, Artisan commands, configuration, and the request lifecycle.

Topic 2 — Project Bootstrapping Workflow: Create a fresh Laravel project, connect routes/controllers/views, use layouts, and manage frontend assets with Vite.

Home Task Problem: Install Laravel and create a multi-page site with shared layout, home, about, services, and contact routes/controllers/views.

Class 15

Migrations, Seeders & Eloquent Basics

Topic 1 — Database Setup in Laravel: Use migrations, seeders, factories, indexes, and foreign keys to create repeatable database setup and realistic sample data.

Topic 2 — Models & Relationships: Use Eloquent models, mass-assignment rules, relationships, route model binding, and basic query methods for application data.

Home Task Problem: Build User, Project, and Task models with relationships, seed sample data, and display project/task records through Laravel.

Phase 4 • Classes 16-20

Phase 4 — Core Laravel Application Development

Turn Laravel fundamentals into secure, structured, and scalable application features.

Class 16

Forms, Validation & CRUD

Topic 1 — Requests, Validation & Feedback: Handle form requests, validation rules, old input, custom messages, and clear validation-error feedback in Laravel.

Topic 2 — Complete CRUD Workflow: Build create, read, update, and delete workflows using resourceful routes/ controllers and consistent naming conventions.

Home Task Problem: Create a complete Projects or Tasks CRUD module with validation, success messages, edit/ update flow, and delete confirmation.

Class 17

Authentication & Middleware

Topic 1 — Authentication Flow: Implement registration, login, logout, sessions, password/reset concepts, and a current Laravel authentication starter workflow.

Topic 2 — Route Protection with Middleware: Protect dashboards and private features, distinguish guest/ authenticated behavior, and keep protected routes grouped clearly.

Home Task Problem: Add authentication so only logged-in users can access and manage the project/task area of the course application.

Class 18

Authorization, Roles & Policies

Topic 1 — Role-Based Access Control: Use policies, gates, middleware, and role-based rules to decide who can view, create, update, or delete resources.

Topic 2 — Ownership & Permission Logic: Enforce authorization on the server so access depends on ownership, assignment, role, or business rules rather than hidden buttons alone.

Home Task Problem: Implement Admin, Manager, and Member permissions and demonstrate blocked unauthorized actions with correct responses.

Class 19

File Uploads & Storage

Topic 1 — Validation & Storage Disks: Validate file type/size, use Laravel Storage disks, understand public/private files, and avoid unsafe filenames or upload handling.

Topic 2 — Attachment User Experience: Upload, display, preview, download, replace, and remove attachments while keeping authorization and validation in place.

Home Task Problem: Add secure task attachment upload, preview/download, replace, and delete functionality with proper validation.

Class 20

Query Optimization & Pagination

Topic 1 — Efficient Data Retrieval: Use eager loading, scopes, relationship counts, filters, sorting, search, and pagination to retrieve useful data efficiently.

Topic 2 — N+1 & Performance Awareness: Recognize N+1 queries, reduce unnecessary database work, and keep list pages scalable as record counts increase.

Home Task Problem: Build a searchable, filterable, sortable, paginated task list and verify related project/user data does not create obvious N+1 queries.

Phase 5 • Classes 21-25

Phase 5 — APIs, React & Inertia

Extend Laravel into modern interactive frontend workflows using APIs, React, and Inertia.js.

Class 21

Building APIs with Laravel

Topic 1 — API Routes, Controllers & Resources: Create API endpoints with clear routes, controllers, validation, status codes, and resource-style JSON responses.

Topic 2 — API Testing Workflow: Use Postman or Insomnia to inspect requests, payloads, validation failures, authentication behavior, and API responses.

Home Task Problem: Build project/task API endpoints and test create, list, update, delete, validation, and error scenarios with an API client.

Class 22

React Fundamentals

Topic 1 — Components, JSX & Props: Understand React component composition, JSX, props, list rendering, conditional rendering, and reusable UI structure.

Topic 2 — State, Events & Hooks: Use component state, events, form inputs, and core hooks to build interactive interfaces without unnecessary complexity.

Home Task Problem: Build reusable dashboard summary widgets plus a project-card list with local interaction state and empty states.

Class 23

Inertia.js with Laravel

Topic 1 — Laravel + React Bridge: Use Inertia pages, props, shared data, links, redirects, flash messages, and Laravel controllers without building an unnecessary separate SPA API.

Topic 2 — Server Data in React Pages: Pass validated/authorized server data to React pages and keep Laravel as the source of routing, validation, and access control.

Home Task Problem: Convert one existing Blade CRUD module to Inertia + React while preserving validation errors, flash messages, and authorization.

Class 24

Advanced Inertia Forms & UX

Topic 1 — Form State & Validation Sync: Use Inertia form helpers, server-side validation errors, state preservation, redirects, and partial reload patterns for smoother forms.

Topic 2 — Filters, Modals & Better UX: Create URL-driven filters, modal create/edit patterns, loading feedback, and consistent error/success states for application workflows.

Home Task Problem: Build a create/edit modal workflow for Projects with validation, filters, persistent state, and clear feedback.

Class 25

Reusable Components & Clean Frontend Structure

Topic 1 — Reusable UI System: Create consistent buttons, inputs, cards, tables, badges, modals, empty states, and page-layout components for repeated use.

Topic 2 — Frontend Organization: Use clear naming, folder boundaries, shared components, and predictable page/feature structure so React code remains maintainable.

Home Task Problem: Create a reusable component library and refactor at least three existing Inertia pages to use the shared components.

Phase 6 • Classes 26-30

Phase 6 — Production, Quality & Final Project

Prepare students to test, secure, document, deploy, and present a production-style Laravel application.

Class 26

Jobs, Events & Notifications

Topic 1 — Background Processing: Use events, listeners, queued jobs, retries, and failure-aware patterns to move expensive work out of the main request cycle.

Topic 2 — Notifications & Business Events: Send email or in-app notifications for meaningful business events such as assignments, status changes, or reminders.

Home Task Problem: When a task is assigned, trigger an event and send a queued notification to the assigned user.

Class 27

Testing, Debugging & Quality Assurance

Topic 1 — Unit & Feature Testing: Write feature tests for HTTP behavior and unit tests for focused business logic; use factories and database reset patterns where appropriate.

Topic 2 — Debugging & Regression Thinking: Use logs, debugging tools, failed-test output, and repeatable test cases to identify issues and prevent regressions.

Home Task Problem: Write automated tests for login, project CRUD, authorization, validation, and task assignment; fix any failing scenarios.

Class 28

Security & Production Readiness

Topic 1 — Application Security Essentials: Review CSRF, XSS, validation, mass assignment, secure file handling, authorization, secret management, and safe production settings.

Topic 2 — Operational Readiness: Prepare caching, logging, backups, database migrations, queue workers, environment variables, HTTPS, and error-handling expectations.

Home Task Problem: Create a production-readiness checklist and fix at least three security or operational issues found in your own project.

Class 29

GitHub, CI/CD & Live Deployment

Topic 1 — GitHub Workflow & CI/CD: Use GitHub repositories, branches, pull requests, protected workflows, and a basic CI/CD pipeline that can automatically run tests and build application assets.

Topic 2 — Server Configuration & Live Deployment: Prepare a Linux/VPS environment, configure web server and PHP runtime, environment variables, database, queues, scheduler, storage, HTTPS, caches, and production deployment steps.

Home Task Problem: Push the capstone to GitHub, prepare a CI/CD workflow, and deploy the application to a live or staging server using a documented server-configuration checklist.

Class 30

Final Capstone, Production Review & Handover

Topic 1 — Production Integration & Verification: Combine all application modules, confirm the live deployment, verify queues, scheduler, storage, database connectivity, environment configuration, and critical user flows.

Topic 2 — Monitoring, Rollback & Professional Handover: Review logs, backups, failure recovery, rollback strategy, security checks, README documentation, demo data, and presentation readiness.

Home Task Problem: Submit the final live URL, GitHub repository, deployment documentation, server checklist, and a walkthrough covering architecture, database design, testing, deployment, challenges, and improvements.

Assessment Plan & Final Evaluation

Six structured assessment classes measure practical progress after each learning stage.

01
Assessment 1
After Class 5
Frontend Foundations
Build a responsive landing page using semantic HTML, CSS, and basic JavaScript with navigation, hero, feature sections, form, and mobile layout.

02
Assessment 2
After Class 10
PHP & OOP Mini App
Create a mini PHP application with server-side form handling, validation, and OOP structure using reusable classes.

03
Assessment 3
After Class 15
Database + Laravel Basics
Design a relational database and build a basic Laravel CRUD module using migrations, seeders, Eloquent, routes, controllers, and Blade.

04
Assessment 4
After Class 20
Secure Laravel Module
Develop an authenticated module with roles/policies, validation, file upload, filters, search, pagination, and optimized data loading.

05
Assessment 5
After Class 25
React + Inertia Integration
Build a Laravel + React + Inertia dashboard module with reusable components, forms, server validation, filters, and feedback states.

06
Assessment 6
After Class 30
Final Capstone Evaluation
Present the capstone, complete a code review and viva, and submit source code, database/seed data, README, and demo-ready delivery materials.

Suggested Grading

- Assignments & Home Tasks - 20%
- Phase Assessments - 30%
- Final Project - 30%
- Viva & Presentation - 10%
- Attendance & Participation - 10%

Final Student Deliverables

- GitHub source-code repository
- Database and sample seed data
- README and installation documentation
- Live deployed capstone URL
- Server/deployment checklist and presentation

Production Technology Skills Included in This Course

GitHub Workflow
Repositories, branches, commits, pull requests, code review, and release-ready source control.

CI/CD Pipeline
Automated tests, asset builds, deployment checks, and repeatable delivery workflow.

Server Configuration
Linux/VPS, web server, PHP runtime, database, queues, scheduler, environment, and HTTPS.

Live Deployment
Production release, migrations, caches, logs, backups, monitoring awareness, and rollback planning.

Final production standard: students should be able to move a tested Laravel + React + Inertia application from a GitHub repository to a configured live server using a documented deployment workflow.

Build -> Test -> GitHub -> CI/CD -> Configure -> Deploy -> Monitor -> Handover
End-to-end production workflow students should understand and demonstrate by the final assessment.